
PERSONAL PORTFOLIO PROJECT

S&P 500 Closing Price Prediction Using LSTM Neural Networks

A Comparative Study of Price-Level and Return-Based
Regression Targets

Gokul Raj Srinivasan

gokulrajs941@gmail.com 07345583896

September 2, 2026

Contents

1	Introduction	6
1.1	Background	6
1.2	Problem Statement	6
1.3	Aim and Objectives	7
1.4	Report Structure	7
2	Background	8
2.1	Long Short-Term Memory Networks	8
2.2	The Efficient Market Hypothesis and Random Walk Behaviour	8
2.3	Technical Indicators as Model Features	9
3	Methodology	10
3.1	Data Acquisition	10
3.2	Feature Engineering	10
3.3	Scaling and Sequence Construction	11
3.4	Train/Test Split	11
3.5	Model Architecture	11
3.5.1	Experiment 1: Absolute Price Prediction	11
3.5.2	Experiment 2: Return-Based Prediction	12
3.6	Baseline Model	12
3.7	Evaluation Metrics	12
3.8	Implementation Setup	12
4	Experimental Results and Analysis	13
4.1	Experiment 1: Absolute Price Prediction	13
4.1.1	Training Dynamics	13

4.1.2	Predicted vs Actual Closing Price	13
4.1.3	Error Metrics and Baseline Comparison	13
4.1.4	Residual Analysis	14
4.1.5	Feature Correlation	14
4.1.6	Directional Accuracy	15
4.2	Experiment 2: Return-Based Prediction	15
4.2.1	Training Dynamics and Mode Collapse	15
4.2.2	Predicted vs Actual Returns	16
4.2.3	Error Metrics and Directional Accuracy	16
4.3	Comparative Summary	17
5	Discussion	19
5.1	Interpretation of Results	19
5.2	Mode Collapse in the Return-Based Model	19
5.3	Practical Implications	20
5.4	Limitations	20
5.5	Future Work	20
6	Conclusion	21

List of Figures

4.1	Predicted vs actual S&P 500 closing price on the held-out test set.	14
4.2	Prediction error (actual – predicted) over the test period.	15
4.3	Correlation heatmap of engineered features.	16
4.4	Confusion matrix for direction (Up/Down) prediction.	17
4.5	Predicted vs actual daily returns on the held-out test set.	17

List of Tables

4.1	Price-level model vs naive baseline: error metrics.	14
4.2	Classification report for directional accuracy (price-level model).	15
4.3	Return-based model: error metrics.	16
4.4	Comparative performance summary.	18

Introduction

1.1 Background

Stock market prediction has long been a subject of interest for both academic researchers and financial practitioners. The S&P 500 index, comprising 500 of the largest publicly traded companies in the United States, is widely used as a benchmark for the overall health of the U.S. equity market. Advances in deep learning, particularly recurrent architectures such as Long Short-Term Memory (LSTM) networks, have been proposed as tools capable of learning temporal dependencies in sequential financial data that traditional statistical models may not capture.

1.2 Problem Statement

Despite considerable interest in applying LSTM networks to financial forecasting, a persistent challenge is that daily price movements in liquid, well-studied markets such as the S&P 500 are close to a random walk, in line with the weak form of the efficient market hypothesis. This raises the question of whether an LSTM model can meaningfully outperform a naive forecast, and whether framing the prediction task differently (absolute price versus percentage return) changes the outcome.

1.3 Aim and Objectives

The aim of this project is to build and rigorously evaluate an LSTM-based forecasting pipeline for the S&P 500 index, with an explicit, honest comparison against a naive baseline. The specific objectives are:

- To construct a complete data pipeline: acquisition, feature engineering, scaling, and sequence generation for time-series modelling.
- To design and train an LSTM regression model predicting the next-day closing price.
- To design and train a second LSTM model predicting the next-day percentage return, as an alternative target formulation.
- To evaluate both models against a naive persistence baseline using regression metrics (RMSE, MAE, MAPE) and a directional-accuracy classification analysis (confusion matrix, precision, recall, F1-score).
- To interpret the results critically in the context of established financial theory.

1.4 Report Structure

Chapter 2 reviews relevant background on LSTM networks and the efficient market hypothesis. Chapter 3 describes the methodology, including data acquisition, feature engineering, and model architecture. Chapter 4 presents the experimental results for both models. Chapter 5 discusses the findings in relation to the literature. Chapter 6 concludes the report.

Background

2.1 Long Short-Term Memory Networks

LSTM networks are a class of recurrent neural network designed to model sequential data by maintaining an internal memory cell that selectively retains or discards information across time steps [1]. This architecture addresses the vanishing gradient problem that limits standard recurrent networks, allowing LSTMs to learn dependencies over longer sequences. LSTMs have been widely applied to time-series forecasting tasks, including financial price prediction, due to their ability to model both short-term fluctuations and longer-term trends within a single architecture.

2.2 The Efficient Market Hypothesis and Random Walk Behaviour

The efficient market hypothesis (EMH) posits that asset prices reflect all available information, such that future price movements cannot be systematically predicted from historical data alone [2]. Under the weak form of the EMH, historical price data contains no exploitable predictive signal, implying that a model's best strategy is close to predicting no change from the current price – equivalent to a random walk. This theoretical framing is directly relevant to the present study, as it provides a lens through which to interpret a model's failure to outperform a naive baseline: such a result is

not necessarily indicative of a flawed implementation, but may instead reflect the underlying statistical properties of the data.

2.3 Technical Indicators as Model Features

Technical indicators are commonly used in quantitative finance to summarise price momentum and trend. The moving average smooths recent price action over a fixed window, while the Relative Strength Index (RSI) measures the magnitude of recent price changes to identify overbought or oversold conditions [3]. These indicators were incorporated as additional input features in this study, on the basis that they may provide the model with more informative signal than raw price alone.

Methodology

3.1 Data Acquisition

Daily historical data for the S&P 500 index (ticker ^GSPC) was retrieved using the `yfinance` Python library, covering a period of five years at daily resolution. The raw dataset comprised Open, High, Low, Close, and Volume (OHLCV) values for 1,254 trading days.

3.2 Feature Engineering

Two technical indicators were computed from the closing price series: a 10-day moving average (MA10) and a 50-day moving average (MA50), capturing short- and medium-term trend respectively. The 14-day Relative Strength Index (RSI) was computed as:

$$RSI = 100 - \frac{100}{1 + RS}, \quad RS = \frac{\text{Average Gain}_{14}}{\text{Average Loss}_{14}} \quad (3.1)$$

Rows containing missing values resulting from the rolling-window calculations (predominantly the first 49 rows, corresponding to the MA50 warm-up period) were removed, leaving 1,205 rows of complete data.

3.3 Scaling and Sequence Construction

All input features were scaled to the range $[0, 1]$ using min-max normalisation, fitted on the full feature set. Input sequences were constructed using a sliding window of 60 trading days (≈ 3 months), such that each sample consists of the preceding 60 days of feature data used to predict the following day's target value. This produced 1,145 total sequences.

3.4 Train/Test Split

Data was split chronologically, with the first 80% of sequences (916 samples) used for training and the most recent 20% (229 samples) held out for testing. A chronological split was used rather than a random split, since random shuffling would allow the model to be trained on data temporally after its test set, violating the causal structure of the forecasting task.

3.5 Model Architecture

Both experiments used an identical LSTM architecture, differing only in the prediction target, to ensure a controlled comparison. The architecture consisted of:

- An LSTM layer with 64 units, returning full sequences to the next layer.
- A second LSTM layer with 32 units, returning a single summary vector.
- A fully connected Dense layer with 16 units and ReLU activation.
- A final Dense output layer with a single unit (linear activation), producing the regression prediction.

The model was compiled with the Adam optimiser and mean squared error (MSE) loss, and trained for 50 epochs with a batch size of 32 and a 10% validation split.

3.5.1 Experiment 1: Absolute Price Prediction

The first model was trained to predict the next-day closing price directly (a continuous regression target), using 8 input features: Close, High, Low, Open, Volume, MA10,

MA50, and RSI.

3.5.2 Experiment 2: Return-Based Prediction

The second model was trained to predict the next-day percentage return, $r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$, using the same 8 features plus the return series itself as a 9th input feature.

3.6 Baseline Model

A naive persistence baseline was defined as predicting that the next day's value equals the current day's actual value. This baseline serves as the minimum bar that a genuinely predictive model should be expected to clear.

3.7 Evaluation Metrics

Model performance was evaluated using:

- **RMSE** (Root Mean Squared Error) and **MAE** (Mean Absolute Error), in index points.
- **MAPE** (Mean Absolute Percentage Error).
- **Directional accuracy**, obtained by converting predicted and actual price changes into a binary Up/Down label and computing a confusion matrix and classification report (precision, recall, F1-score).

3.8 Implementation Setup

All models were implemented in Python using TensorFlow/Keras, with `scikit-learn` used for scaling and evaluation metrics, and `pandas/numpy` for data manipulation. Experiments were run locally on CPU.

Experimental Results and Analysis

4.1 Experiment 1: Absolute Price Prediction

4.1.1 Training Dynamics

Training loss decreased steadily from 0.0079 to approximately 0.00033 over 50 epochs, with validation loss following a similar downward trend, ending at approximately 0.00026. The proximity of final training and validation loss suggests the model was not substantially overfitting to the training data.

4.1.2 Predicted vs Actual Closing Price

Figure 4.1 shows the model's predictions against actual closing prices on the held-out test set. While the predicted series tracks the general shape of the actual trend, it consistently underestimates the actual value, particularly during the sustained upward rally in the latter half of the test period.

4.1.3 Error Metrics and Baseline Comparison

Table 4.1 summarises the error metrics for the price-level model against the naive baseline.

The naive baseline outperforms the LSTM model by a substantial margin across all three metrics, with MAE more than four times lower.

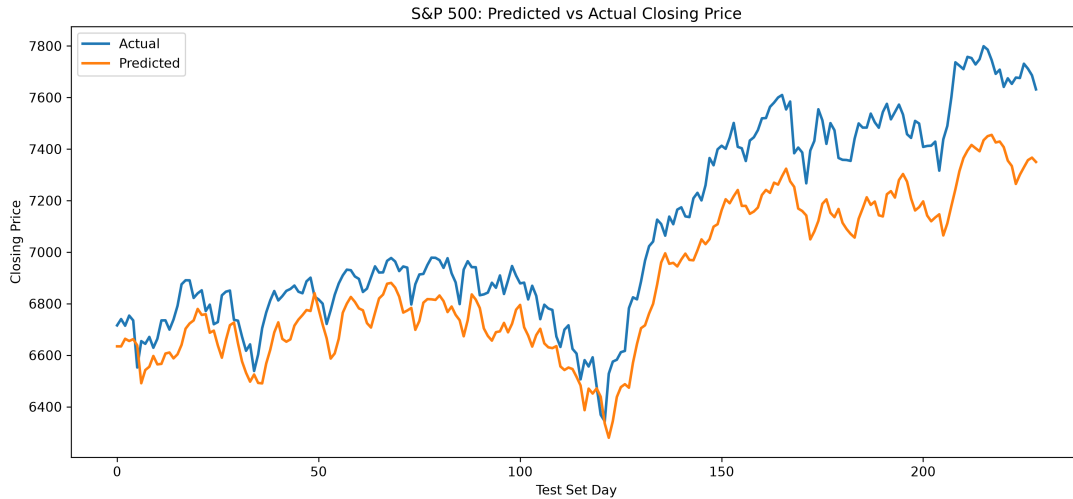


Figure 4.1: Predicted vs actual S&P 500 closing price on the held-out test set.

Table 4.1: Price-level model vs naive baseline: error metrics.

Model	RMSE (points)	MAE (points)	MAPE
LSTM (price regression)	221.32	196.09	2.72%
Naive baseline	58.50	44.69	0.64%

4.1.4 Residual Analysis

Figure 4.2 plots the prediction error (actual – predicted) over the test period. The residuals sit almost entirely above zero, confirming a systematic underprediction bias, and the magnitude of the error increases over time as the index trends upward – consistent with a model that lags behind genuine directional moves rather than anticipating them.

4.1.5 Feature Correlation

Figure 4.3 shows the pairwise correlation between all engineered features. As expected, Close, High, Low, Open, MA10, and MA50 are all highly correlated (0.99–1.00), reflecting their shared basis in price. Volume shows only weak correlation with price (0.33–0.39), while RSI is largely independent of the price-based features (0.15–0.20), confirming it contributes distinct information to the model.

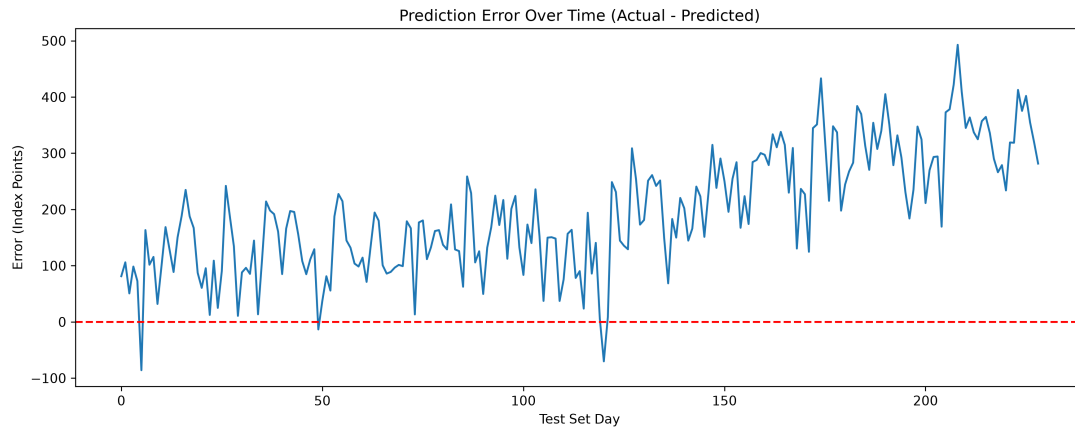


Figure 4.2: Prediction error (actual – predicted) over the test period.

4.1.6 Directional Accuracy

Converting predictions to a binary Up/Down direction label, Figure 4.4 and Table 4.2 show the model achieved only 48% directional accuracy – statistically indistinguishable from a coin flip.

Table 4.2: Classification report for directional accuracy (price-level model).

Class	Precision	Recall	F1-score	Support
Down	0.43	0.42	0.43	104
Up	0.52	0.53	0.53	124
Accuracy		0.48		228

4.2 Experiment 2: Return-Based Prediction

4.2.1 Training Dynamics and Mode Collapse

Training loss for the return-based model dropped sharply within the first epoch (0.0340 → 0.0067) but then plateaued at approximately 0.0059–0.0060 for the remaining 49 epochs, showing negligible further improvement. This flat trajectory is an early indicator of mode collapse, whereby the model settles on predicting a value close to the target’s mean rather than learning genuine input-dependent patterns.



Figure 4.3: Correlation heatmap of engineered features.

4.2.2 Predicted vs Actual Returns

This hypothesis is confirmed in Figure 4.5. Actual daily returns range between -2.7% and $+2.9\%$, while predicted returns are confined to a narrow band between -0.34% and -0.08% (standard deviation of 0.00054, compared to 0.00832 for actual returns) – a range roughly 15 times narrower than the true variability in the data.

4.2.3 Error Metrics and Directional Accuracy

Table 4.3 summarises the return-based model's performance. Directional accuracy of 45.41% is not only close to chance but slightly worse than the price-level model.

Table 4.3: Return-based model: error metrics.

Metric	Value
MAE	0.00676
RMSE	0.00871
Directional Accuracy	45.41%

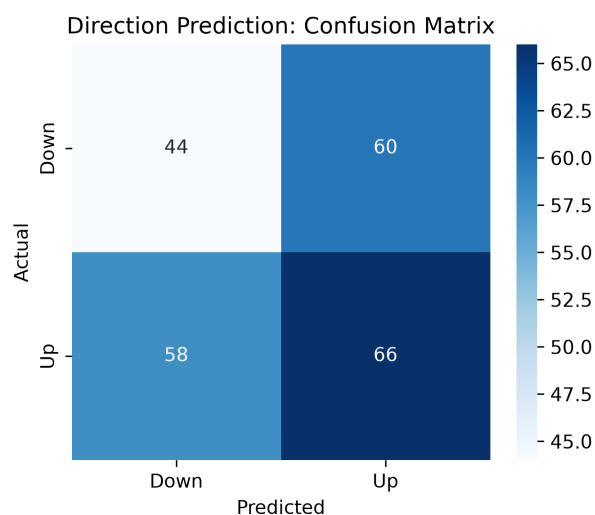


Figure 4.4: Confusion matrix for direction (Up/Down) prediction.

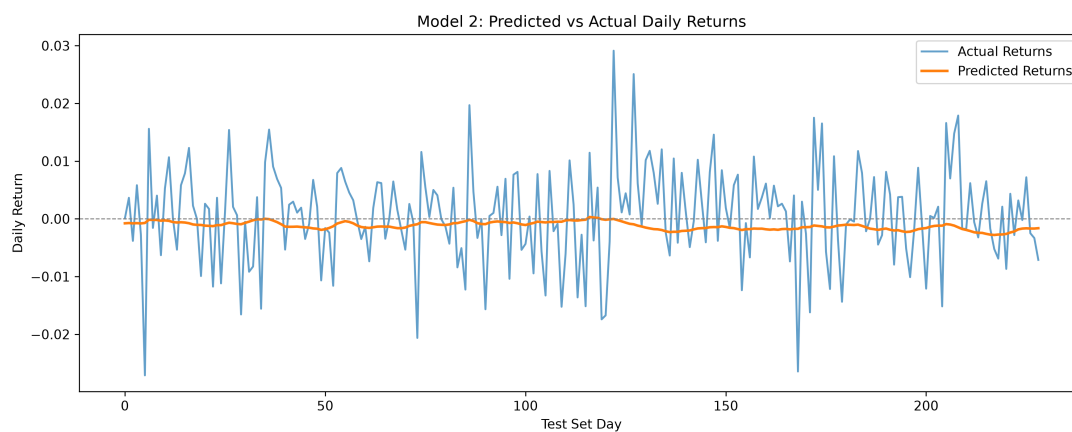


Figure 4.5: Predicted vs actual daily returns on the held-out test set.

4.3 Comparative Summary

Table 4.4 consolidates the results across both experiments and the naive baseline.

Table 4.4: Comparative performance summary.

Model	Error (relative)	Directional Accuracy
Naive baseline	Best (MAE 44.69 pts)	–
LSTM (price regression)	4.4× worse than baseline	48%
LSTM (return regression)	Mode collapse	45%

Discussion

5.1 Interpretation of Results

Neither model formulation outperformed the naive persistence baseline, and both achieved directional accuracy statistically close to chance. This outcome is consistent with the weak-form efficient market hypothesis: since the S&P 500 is one of the most liquid, heavily analysed indices in the world, any short-term predictive signal exploitable from historical price and volume data alone is likely to have already been arbitrated away by market participants. The price-level model's tendency to lag the true series is a well-documented behaviour in LSTM price forecasting, arising because predicting a value close to the recent price minimises mean squared error even without learning genuine predictive structure.

5.2 Mode Collapse in the Return-Based Model

The return-based model's collapse to near-constant, near-zero predictions illustrates a further consequence of attempting to predict an inherently noisier target. Because daily returns are dominated by short-term noise with a mean close to zero, the loss-minimising strategy for the optimiser was to output a value close to that mean, rather than attempting to model genuine variation. This is a cautionary finding for future work: reformulating a forecasting target to remove an easy shortcut (as returns do,

relative to price) does not guarantee a more informative model, and can instead expose the model to a different, more severe form of underfitting.

5.3 Practical Implications

These findings have direct implications for how LSTM-based price prediction should be positioned in a portfolio context. Rather than treating a low training loss as evidence of a useful trading signal, this study demonstrates the necessity of baseline comparison: a model can appear to train successfully (smooth, converging loss curves) while providing no practical predictive value beyond a trivial baseline. This distinction is particularly important given the prevalence of LSTM stock-prediction tutorials that report loss curves without a naive baseline for comparison, potentially overstating the model's real-world utility.

5.4 Limitations

Several limitations should be noted. First, the model relies solely on historical price, volume, and technical indicators; it does not incorporate external information such as news sentiment, macroeconomic indicators, or order-book data, any of which might carry more genuine predictive signal. Second, the 5-year training window, while sufficient for model convergence, spans a limited range of market regimes. Third, the study evaluates only a single architecture (a 2-layer LSTM); alternative architectures, loss functions (e.g. directly optimising for directional accuracy), or ensemble approaches were not explored.

5.5 Future Work

Future work could explore classification-first approaches that directly optimise for directional accuracy rather than deriving it from a regression output, incorporate external features such as macroeconomic indicators or news sentiment, or apply attention-based architectures (e.g. Transformers) that have shown promise in other financial forecasting contexts. A systematic comparison against classical statistical baselines such as ARIMA or GARCH would also strengthen the evaluation.

Conclusion

This report developed and rigorously evaluated two LSTM-based approaches to forecasting the S&P 500 index: one predicting absolute closing price, and one predicting daily percentage returns. Both approaches were benchmarked against a naive persistence baseline and evaluated using regression error metrics and directional-accuracy classification analysis. Neither LSTM model outperformed the naive baseline, and both achieved directional accuracy statistically indistinguishable from chance, with the return-based model additionally exhibiting mode collapse to near-constant predictions.

Rather than representing a failed project, these results constitute a legitimate and evidence-based empirical finding, consistent with the weak-form efficient market hypothesis. The primary contribution of this work is not a profitable trading model, but a complete, honest, and reproducible evaluation pipeline that correctly identifies the limits of LSTM-based forecasting on a single, well-studied financial time series – a distinction that is often overlooked in tutorial-style treatments of this problem, and one that reflects sound practice in applied machine learning evaluation.

Bibliography

- [1] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [2] E. F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.
- [3] J. W. Wilder. *New Concepts in Technical Trading Systems*. Trend Research, Greensboro, NC, 1978.